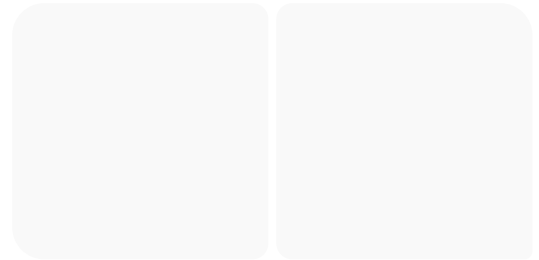


Seminario de actualización en la nube

no aparece

donde?

ya no aparecen errores



Sí, y de hecho ahora que ya lo hiciste una vez, tiene mucho sentido documentarlo porque este procedimiento te sirve para:

- React + Servlet
- Spring Boot
- PHP
- Python Flask
- FastAPI
- Proyectos meteorológicos Java/Python
- APIs REST en general

La idea sería pensarlo como una **guía general de despliegue moderno**.

Flujo general

```
Desarrollo
  ↓
Docker
  ↓
GitHub
  ↓
GitHub Actions
  ↓
AWS Elastic Beanstalk
  ↓
Internet
```

FASE 1 - Desarrollo local

1. Crear la aplicación

Ejemplo:

Frontend React
Backend Servlet

Probar:

http://localhost:5173

http://localhost:8080/backend-servlet/api/alumnos

FASE 2 - Dockerización

Backend

Dockerfile:

```
</> dockerfile
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn clean package

FROM tomcat:10.1-jdk17

COPY --from=build /app/target/backend-servlet.war \
/usr/local/tomcat/webapps/backend-servlet.war

EXPOSE 8080
```

Frontend

Dockerfile:

</> dockerfile

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist \
/usr/share/nginx/html

COPY nginx.conf \
/etc/nginx/conf.d/default.conf

EXPOSE 80
```

nginx.conf

</> Nginx

```
server {

    listen 80;

    location / {

        root /usr/share/nginx/html;
        index index.html;
        try_files $uri /index.html;

    }

    location /backend-servlet/ {

        proxy_pass http://backend:8080/backend-servlet/;

    }

}
```

React

Nunca:

```
</> JavaScript
fetch("http://localhost:8080/...")
```

Siempre:

```
</> JavaScript
fetch("/backend-servlet/api/alumnos")
```

FASE 3 - Docker Compose

```
</> YAML
services:

  backend:

    build: ./backend-servlet

    ports:
      - "8080:8080"

  frontend:

    build: ./frontend-react

    ports:
      - "80:80"

    depends_on:
      - backend
```

Probar

```
</> Bash
docker compose up --build
```

Verificar:

http://localhost

FASE 4 - GitHub

Crear repositorio

</> Bash

```
git init

git add .

git commit -m "Version inicial"

git branch -M main

git remote add origin URL

git push -u origin main
```

FASE 5 - CI

Crear workflow

Ruta:

.github/workflows/ci.yml

Workflow:

```
Push
↓
Compilar backend
↓
Compilar frontend
```

Verificar

</> Bash

```
git push origin main
```

GitHub:

Actions

Debe quedar:

✓ build backend

✓ build frontend

FASE 6 - AWS

Crear entorno Elastic Beanstalk

Buscar:

Elastic Beanstalk

Configuración

Servidor Web

Docker

Instancia única

t3.micro

Crear roles IAM

Servicio

```
aws-elasticbeanstalk-service-role
```

Políticas:

```
AWS ElasticBeanstalkEnhancedHealth
```

```
AWS ElasticBeanstalkManagedUpdatesCustomerRolePolicy
```

EC2

```
aws-elasticbeanstalk-ec2-role
```

Políticas:

```
AWS ElasticBeanstalkWebTier
```

```
AWS ElasticBeanstalkWorkerTier
```

```
AWS ElasticBeanstalkMulticontainerDocker
```

Crear entorno

Esperar:

```
Status: OK
```

Obtener URL:

```
https://xxxxx.elasticbeanstalk.com
```

FASE 7 - Credenciales AWS

IAM

```
Usuario:  
github-actions
```

Permiso:

AdministratorAccess

Crear Access Key

Generar:

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

FASE 8 - GitHub Secrets

Repositorio:

Settings

↓

Secrets and Variables

↓

Actions

Crear:

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

FASE 9 - CD

Workflow completo:

Push

↓

Compila backend

↓

Compila frontend

↓

Genera ZIP

↓
Deploy AWS

Resultado

</> Bash

```
git add .  
  
git commit -m "Cambio"  
  
git push origin main
```

Automáticamente:

GitHub Actions
↓
CI
↓
CD
↓
AWS
↓
Aplicación actualizada

FASE 10 - Operación diaria

Flujo de trabajo normal:

Modificar código
↓
Probar localmente
↓
docker compose up --build
↓
Verificar
↓
git add .
↓
git commit
↓
git push origin main
↓
GitHub Actions

↓
AWS

Y listo.

A partir de ahora, cualquier proyecto que hagas (por ejemplo tu generador de cartas GFS en Java/Python) puede seguir exactamente el mismo esquema:

```
Proyecto
  ↓
Docker
  ↓
GitHub
  ↓
GitHub Actions
  ↓
AWS
```

cambiando únicamente el contenido de los Dockerfile. Esa es justamente la gran ventaja de Docker y CI/CD: el proceso de despliegue se vuelve casi idéntico independientemente de la tecnología que uses.



Vamos a generar un trabajo práctico con: 1era parte generar app con MVC pagina jsp generada por el servidor y servlet, osea aplicación monolítica... ya lo hicimos esto... pero lo convertimos en la primera parte que aún no hice con ellos. Se puede usar un Docker en el backend que suba el war y lo ejecute internamente. 2da parte, pasamos a la misma aplicación con REACT en el front, NGINX servidor del front, docker en front y back, docker compose. El back sigue igual. 3era parte generar el CI/CD con GITHUB y el deploy a AWS Elastic. Quiero que ellos intenten hacer el paso a paso y mostrarlo

Sí. Te lo ordenaría como **TP integrador en 3 etapas**, mostrando la evolución: monolito → frontend desacoplado → CI/CD + cloud.



Trabajo Práctico Integrador

De una aplicación monolítica MVC a una aplicación desacoplada con Docker, CI/CD y AWS

Seminario de Actualización

Objetivo general

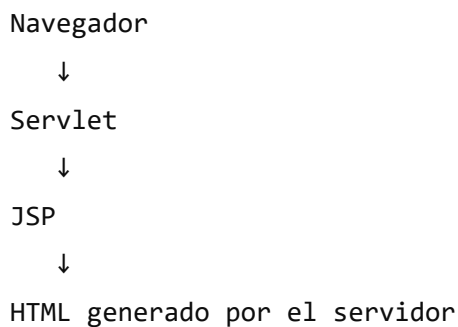
El objetivo de este trabajo práctico es que el estudiante comprenda la evolución de una aplicación web tradicional hacia un esquema moderno de despliegue utilizando frontend desacoplado, contenedores Docker, GitHub Actions y servicios en la nube.

Parte 1: Aplicación monolítica MVC con Servlet y JSP

Objetivo

Construir una aplicación web Java tradicional donde el servidor genere la página HTML mediante JSP.

Arquitectura



Requisitos

La aplicación deberá permitir:

1. Visualizar un listado de alumnos.
2. Agregar un nuevo alumno mediante un formulario.
3. Mostrar los datos usando una página JSP.
4. Utilizar un Servlet como controlador.
5. Mantener los datos en memoria mediante una colección Java.

Estructura sugerida

```
mvc-servlet-jsp
|
├─ src/main/java
|   ├─ modelo
|   │   └─ Alumno.java
|   └─ servlet
|       └─ AlumnoServlet.java
|
├─ src/main/webapp
|   └─ alumnos.jsp
|
├─ pom.xml
└─ Dockerfile
```

Dockerización del backend monolítico

El proyecto deberá generar un archivo `.war` y ejecutarse dentro de un contenedor Tomcat.

Dockerfile sugerido:

```
FROM maven:3.9-eclipse-temurin-17 AS build
```

```
WORKDIR /app
```

```
COPY pom.xml .
```

```
COPY src ./src
```

```
RUN mvn clean package
```

```
FROM tomcat:10.1-jdk17
```

```
COPY --from=build /app/target/mvc-servlet-jsp.war
/usr/local/tomcat/webapps/mvc-servlet-jsp.war
```

```
EXPOSE 8080
```

Prueba local

```
docker build -t mvc-servlet-jsp .
```

```
docker run -p 8080:8080 mvc-servlet-jsp
```

URL de prueba:

<http://localhost:8080/mvc-servlet-jsp/alumnos>

Entrega de la Parte 1

El estudiante deberá entregar:

- Código fuente del proyecto MVC.
- Servlet funcionando.
- JSP funcionando.
- Dockerfile.
- Captura de la aplicación ejecutándose desde Docker.
- Breve explicación de por qué esta aplicación es monolítica.

Parte 2: Aplicación desacoplada con React + Servlet + Docker Compose

Objetivo

Transformar la aplicación anterior en una arquitectura desacoplada, separando frontend y backend.

Arquitectura

Navegador



Frontend React



Backend Servlet



JSON

Requisitos

1. El backend seguirá siendo Java Servlet.
2. El backend ya no devolverá JSP, sino JSON.
3. El frontend será desarrollado con React.
4. React consumirá los datos mediante `fetch`.
5. El frontend será servido con Nginx.
6. Se crearán dos contenedores:
 - uno para el backend;
 - uno para el frontend.

7. Ambos contenedores deberán ejecutarse mediante Docker Compose.

Estructura sugerida

```
app-react-servlet
|
├── backend-servlet
|   ├── src/main/java
|   ├── pom.xml
|   └── Dockerfile
|
├── frontend-react
|   ├── src
|   ├── package.json
|   ├── Dockerfile
|   └── nginx.conf
|
└── docker-compose.yml
```

Backend

El backend deberá exponer el endpoint:

GET /backend-servlet/api/alumnos

y devolver:

```
[
  { "id": 1, "nombre": "Ana" },
  { "id": 2, "nombre": "Juan" },
  { "id": 3, "nombre": "María" }
]
```

También deberá permitir agregar alumnos mediante:

POST /backend-servlet/api/alumnos

Frontend

El frontend deberá:

1. Mostrar el listado de alumnos.
2. Permitir agregar un nuevo alumno.
3. Usar rutas relativas para comunicarse con el backend.

Ejemplo:

```
fetch("/backend-servlet/api/alumnos")
```

No se debe usar:

```
fetch("http://localhost:8080/backend-servlet/api/alumnos")
```

Nginx

El archivo `nginx.conf` deberá funcionar como servidor del frontend y como proxy hacia el backend.

```
server {  
    listen 80;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
        try_files $uri /index.html;  
    }  
  
    location /backend-servlet/ {  
        proxy_pass http://backend:8080/backend-servlet/;  
    }  
}
```

Dockerfile del frontend

```
FROM node:20-alpine AS build
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
RUN npm run build
```

```
FROM nginx:alpine
```

```
COPY --from=build /app/dist /usr/share/nginx/html
```



```
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

```
EXPOSE 80
```

docker-compose.yml

```
services:
```

```
  backend:
```

```
    build: ./backend-servlet
```

```
    ports:
```

```
      - "8080:8080"
```

```
  frontend:
```

```
    build: ./frontend-react
```

```
    ports:
```

```
      - "80:80"
```

```
    depends_on:
```

```
      - backend
```

Prueba local

Desde la raíz del proyecto:

```
docker compose up --build
```

URLs de prueba:

```
http://localhost
```

```
http://localhost:8080/backend-servlet/api/alumnos
```

Entrega de la Parte 2

El estudiante deberá entregar:

- Backend Servlet funcionando como API REST.
- Frontend React consumiendo el backend.
- Dockerfile del backend.
- Dockerfile del frontend.
- Archivo `nginx.conf`.
- Archivo `docker-compose.yml`.
- Captura del frontend funcionando.

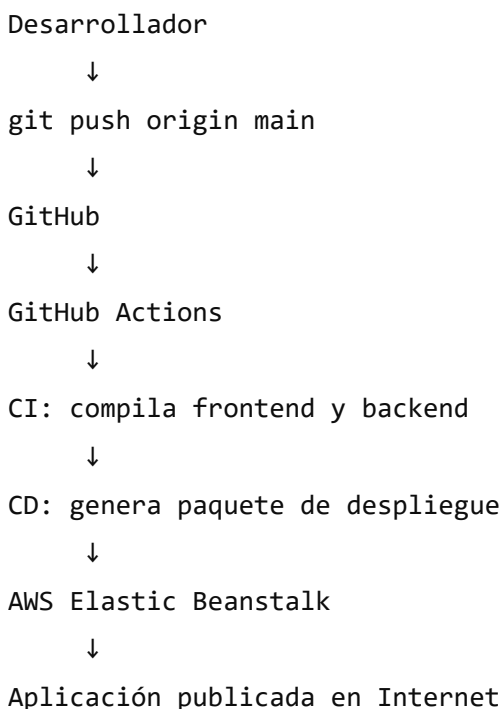
- Captura del endpoint JSON funcionando.
 - Explicación de la diferencia entre aplicación monolítica y aplicación desacoplada.
-

Parte 3: CI/CD con GitHub Actions y despliegue en AWS Elastic Beanstalk

Objetivo

Automatizar el proceso de compilación y despliegue de la aplicación utilizando GitHub Actions y AWS Elastic Beanstalk.

Arquitectura final



Requisitos

1. Subir el proyecto completo a GitHub.
2. Crear un workflow de GitHub Actions.
3. Compilar automáticamente el backend.
4. Compilar automáticamente el frontend.
5. Generar un paquete `.zip`.
6. Desplegar automáticamente en AWS Elastic Beanstalk.
7. Verificar que la aplicación quede publicada en una URL pública.

Configuración en AWS

El estudiante deberá crear:

Elastic Beanstalk

- Tipo de entorno: servidor web.
- Plataforma: Docker.
- Tipo de entorno: instancia única.
- Tipo de instancia sugerido: `t3.micro` o `t2.micro`.
- Base de datos: deshabilitada.

Roles IAM necesarios

Rol de servicio:

`aws-elasticbeanstalk-service-role`

Perfil de instancia EC2:

`aws-elasticbeanstalk-ec2-role`

Usuario IAM para GitHub Actions

Crear un usuario:

`github-actions`

Generar:

`AWS_ACCESS_KEY_ID`

`AWS_SECRET_ACCESS_KEY`

Cargar ambos valores en GitHub:

Repository

↓

Settings

↓

Secrets and variables

↓

Actions

Workflow sugerido

Archivo:

`.github/workflows/ci.yml`

Contenido:

name: CI CD Fullstack React Servlet

on:

push:

branches: ["main"]

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout
uses: actions/checkout@v4
- name: Configurar Java
uses: actions/setup-java@v4
with:
distribution: temurin
java-version: 17
- name: Compilar backend
working-directory: backend-servlet
run: mvn clean package
- name: Configurar Node
uses: actions/setup-node@v4
with:
node-version: 20
- name: Instalar dependencias frontend
working-directory: frontend-react
run: npm install
- name: Compilar frontend
working-directory: frontend-react
run: npm run build
- name: Crear ZIP para Elastic Beanstalk
run: |

```
zip -r deploy.zip . \
-x "*.git*" \
-x "*/node_modules/*"

- name: Deploy AWS Elastic Beanstalk
  uses: einaregilsson/beanstalk-deploy@v22
  with:
    aws_access_key: ${ secrets.AWS_ACCESS_KEY_ID }
    aws_secret_key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    application_name: seminario-actualizacion
    environment_name: seminario-actualizacion-env
    region: us-east-2
    version_label: ${ github.sha }
    deployment_package: deploy.zip
```

Prueba del pipeline

Modificar algún texto o agregar un alumno de prueba.

Luego ejecutar:

```
git add .
git commit -m "Prueba de despliegue automatico"
git push origin main
```

Verificar en GitHub:

Actions

El workflow deberá quedar en verde.

Luego verificar en AWS:

Elastic Beanstalk

↓

Eventos

Debe aparecer un mensaje indicando que la nueva versión fue desplegada correctamente.

Entrega de la Parte 3

El estudiante deberá entregar:

- URL del repositorio en GitHub.

- Captura del workflow ejecutado correctamente.
- Captura de Elastic Beanstalk con estado OK.
- URL pública de la aplicación desplegada.
- Explicación del flujo CI/CD.
- Explicación de qué ocurre cuando se ejecuta `git push origin main`.

Preguntas de análisis

Responder brevemente:

1. ¿Por qué la primera aplicación se considera monolítica?
2. ¿Qué cambia cuando se separa el frontend del backend?
3. ¿Qué función cumple Docker?
4. ¿Qué diferencia hay entre imagen y contenedor?
5. ¿Para qué sirve Docker Compose?
6. ¿Qué función cumple Nginx en la segunda parte?
7. ¿Por qué no conviene usar `localhost` en el `fetch` cuando la aplicación está desplegada?
8. ¿Qué significa CI?
9. ¿Qué significa CD?
10. ¿Qué servicio de AWS se utilizó para publicar la aplicación?
11. ¿Qué ocurre automáticamente cuando se ejecuta `git push origin main`?
12. ¿Qué ventajas tiene este modelo frente a subir archivos manualmente a un servidor?

Criterios de evaluación

Criterio	Puntaje
Aplicación MVC con Servlet y JSP funcionando	15
Dockerización correcta del monolito	10
Backend Servlet funcionando como API JSON	15
Frontend React funcionando	15

Uso correcto de Nginx	10
Docker Compose funcionando	10
Workflow CI/CD funcionando	15
Despliegue exitoso en AWS	10
Total	100

Conclusión esperada

Al finalizar el trabajo práctico, el estudiante deberá poder explicar la evolución desde una aplicación web tradicional hasta una aplicación moderna desplegada en la nube, comprendiendo el rol de Servlet, JSP, React, Docker, Nginx, GitHub Actions y AWS Elastic Beanstalk.



devolveme la teoría para explicar en clase para: Servlet, JSP, WAR, React, Docker, Tomcat, Nginx, CI/CD con GITHUB Actions, AWS Elastic Beanstalk y los temas adicionales que hagan falta para vincular los temas

Para esta clase conviene contar una **historia evolutiva**. No explicar las tecnologías aisladas, sino mostrar cómo una lleva naturalmente a la otra.

1. Aplicaciones Web Tradicionales

Problema a resolver

Queremos mostrar información en un navegador.

Por ejemplo:

Listado de alumnos

Arquitectura:

```
graph TD;
  Navegador --> Servidor;
  Servidor --> Base_datos[Base de datos];
```

2. Servlet

¿Qué es?

Un Servlet es una clase Java que recibe solicitudes HTTP y genera respuestas.

Ejemplo:

GET /alumnos

Llega al Servlet:

```
</> Java
public class AlumnoServlet extends HttpServlet
```

y el Servlet decide qué hacer.

Función

Actúa como controlador.

```
graph TD;
  Navegador --> Servlet;
```

Recibe:

GET
POST
PUT
DELETE

y responde.

Ventaja

Permite centralizar la lógica.

Problema

Generar HTML dentro de Java es incómodo.

Ejemplo:

 Java

```
out.println("<html>");  
out.println("<body>");
```

3. JSP

¿Qué es?

Java Server Pages.

Permite escribir HTML mezclado con Java.

Ejemplo:

 JSP

```
<h1>Listado</h1>  
  
<%  
for(Alumno a : alumnos){  
%>  
  
<p><%= a.getNombre() %></p>  
  
<%  
}  
%>
```

Arquitectura MVC

Navegador



Servlet



JSP



HTML

Rol

Servlet

Controlador

JSP

Vista

Modelo

Clases Java

Alumno

Curso

Materia

4. WAR

¿Qué es?

Web Archive.

Es el equivalente a un JAR para aplicaciones web.

Contiene:

Servlets

JSP

HTML

CSS

JavaScript

Librerías

Ejemplo

miapp.war

¿Quién lo ejecuta?

Tomcat.

5. Tomcat

¿Qué es?

Servidor de aplicaciones Java.

Su función:

```
Tomar un WAR
  ↓
Desplegarlo
  ↓
Atender peticiones HTTP
```

Flujo

```
miapp.war
  ↓
Tomcat
  ↓
Aplicación funcionando
```

Analogía

WAR = programa

Tomcat = consola que lo ejecuta

6. Limitaciones del MVC tradicional

En los años 2000 funcionaba perfecto.

Pero comenzaron a surgir problemas:

- Interfaces complejas
- Mucho JavaScript
- Aplicaciones más dinámicas

Cada cambio requiere:

```
Navegador
  ↓
Servidor
  ↓
Genera HTML nuevamente
```

7. APIs REST

La idea moderna:

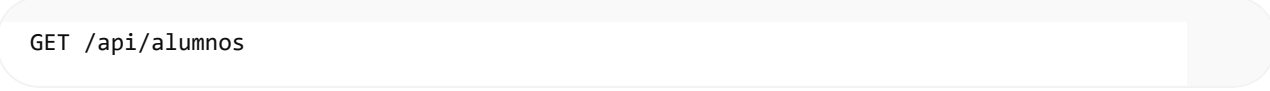
```
El servidor ya no genera HTML
```

Genera datos.

Ejemplo:

```
</> JSON
[
  {
    "id": 1,
    "nombre": "Ana"
  }
]
```

El backend pasa a devolver JSON



```
GET /api/alumnos
```

8. React

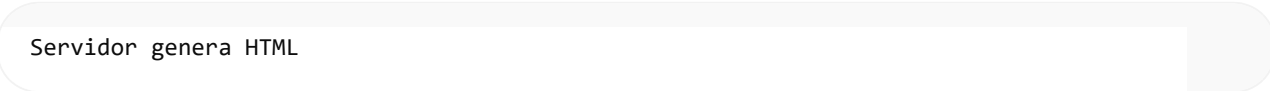
¿Qué es?

Biblioteca JavaScript para construir interfaces.

Idea

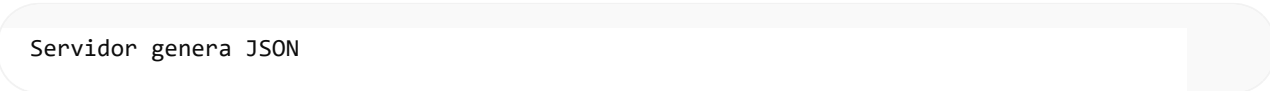
Ahora el HTML se genera en el navegador.

Antes:

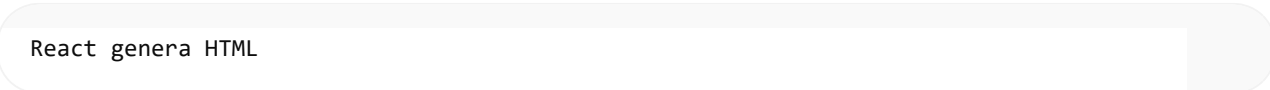


```
Servidor genera HTML
```

Ahora:

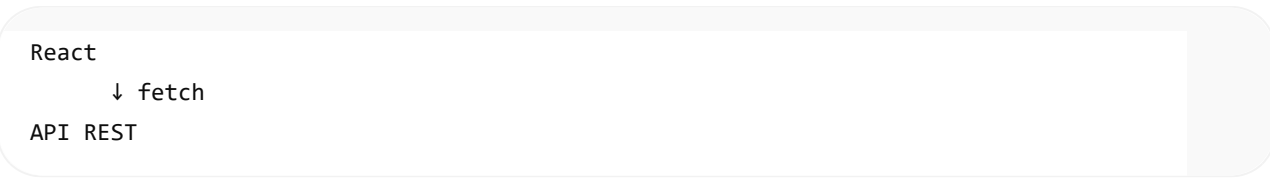


```
Servidor genera JSON
```



```
React genera HTML
```

Arquitectura



```
React  
  ↓ fetch  
API REST
```

Ventaja

Frontend y backend independientes.

9. Frontend desacoplado

Ahora tenemos:

Frontend React

Backend Servlet

Comunicación

⌘ JavaScript

```
fetch("/api/alumnos")
```

Beneficio

Pueden evolucionar por separado.

10. Docker

Problema

En mi máquina funciona.

En otra:

No funciona

Solución

Empaquetar todo.

Código

Java

Tomcat

Configuración

dentro de una imagen.

Imagen

Plantilla.

Docker Image

Contenedor

Instancia ejecutándose.

Docker Container

Analogía

Imagen = plano

Contenedor = casa construida

11. Docker Compose

Problema

Ahora tenemos:

Frontend

Backend

Dos contenedores.

Solución

Docker Compose.

</> YAML

```
services:
  frontend
  backend
```

Ejecución

```
</> Bash
```

```
docker compose up
```

12. Nginx

¿Qué es?

Servidor web.

Función

Entregar archivos.

```
HTML
```

```
CSS
```

```
JS
```

En React

Generamos:

```
</> Bash
```

```
npm run build
```

Obtengo:

```
dist/
```

Nginx publica esos archivos

```
Internet
```

```
↓
```

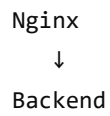
```
Nginx
```

```
↓
```

```
React
```


Además

Puede actuar como Proxy.



```
graph TD; Nginx --> Backend;
```

Nginx
↓
Backend

13. Git

Problema

Necesitamos controlar versiones.

Solución

Git.



```
</> Bash  
git commit
```

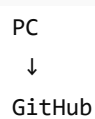
Beneficio

Historial.

Trabajo colaborativo.

14. GitHub

Repositorio remoto.



```
graph TD; PC --> GitHub;
```

PC
↓
GitHub

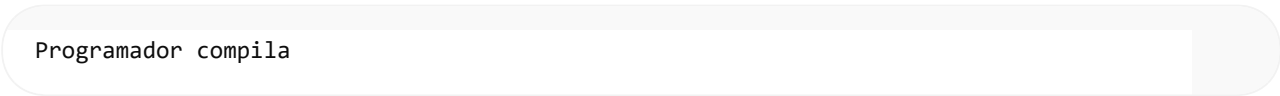
15. CI

Continuous Integration.

Idea

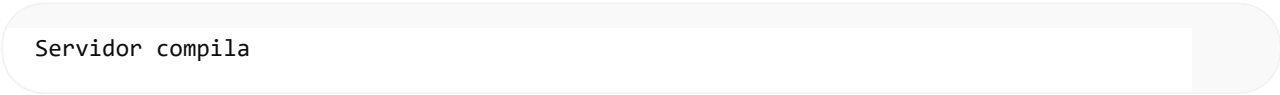
Cada cambio debe verificarse.

Antes:



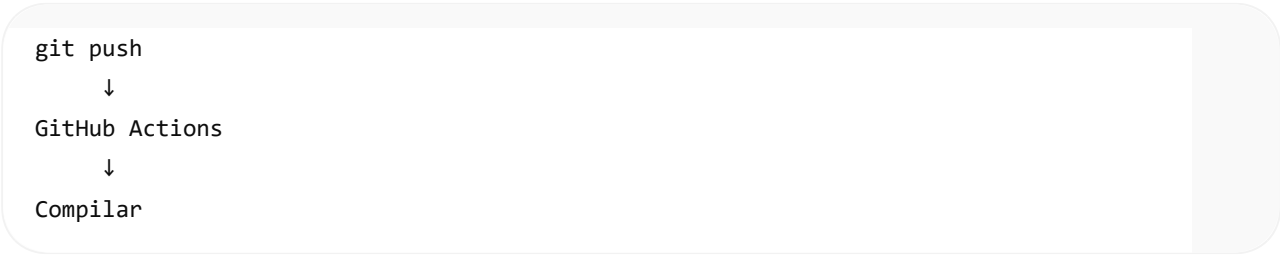
Programador compila

Ahora:



Servidor compila

Flujo

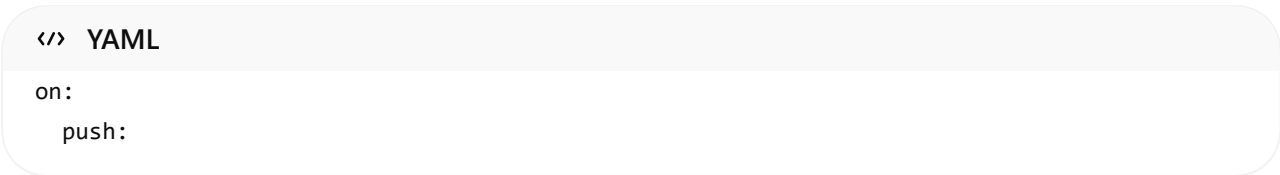


```
graph TD; A[git push] --> B[GitHub Actions]; B --> C[Compilar];
```

16. GitHub Actions

Automatización.

Workflow



```
</> YAML
```

```
on:
  push:
```

Qué hace

```
Compila backend  
Compila frontend  
Ejecuta tests
```

17. CD

Continuous Delivery / Deployment.

Idea

No sólo verificar.

También desplegar.

```
git push  
    ↓  
CI  
    ↓  
Deploy
```

18. Cloud Computing

Problema

Mi PC debe estar encendida.

Solución

Nube.

Beneficio

Servidor disponible:

```
24x7
```

19. AWS

Amazon Web Services.

Servicios:

- EC2
- S3
- RDS
- Lambda
- Elastic Beanstalk

20. Elastic Beanstalk

¿Qué es?

Servicio PaaS.

Platform as a Service.

Idea

AWS administra:

- Servidor
- Sistema operativo
- Tomcat
- Docker

Nosotros administramos:

- Aplicación

Flujo

```
graph TD; ZIP --> ElasticBeanstalk[Elastic Beanstalk]; ElasticBeanstalk --> EC2; EC2 --> Docker; Docker --> Aplicación;
```

ZIP
↓
Elastic Beanstalk
↓
EC2
↓
Docker
↓
Aplicación

21. Arquitectura final

Etapa 1

MVC

```
graph TD; Browser --> Servlet; Servlet --> JSP;
```

Browser
↓
Servlet
↓
JSP

Etapa 2

Desacoplada

```
graph TD; React --> ServletAPI[Servlet API];
```

React
↓
Servlet API

Etapa 3

Contenedores

```
graph TD; React; Servlet; Docker; Compose;
```

React
Servlet
Docker
Compose

Etapa 4

CI/CD

GitHub
↓
Actions

Etapa 5

Cloud

AWS
↓
Elastic Beanstalk

Resumen final para los alumnos

Servlet
↓
JSP
↓
WAR
↓
Tomcat
↓
React
↓
Docker
↓
Docker Compose
↓
Nginx
↓
Git
↓
GitHub
↓
CI/CD
↓
GitHub Actions
↓
AWS



Elastic Beanstalk

Esa secuencia cuenta la evolución completa de una aplicación web desde el modelo clásico Java EE hasta un despliegue moderno en la nube.

